

Memory Management

These notes are written for placement preparation. The goal is to understand the ideas clearly enough to answer theory questions, solve numericals, and explain trade-offs in interviews.

1. What Is Memory Management?

Memory management is the operating system activity that controls how main memory, also called RAM, is allocated, used, protected, shared, and freed among running processes.

When multiple programs run at the same time, each program needs memory for:

- Program code, also called text section
- Global and static variables
- Heap memory, used for dynamic allocation
- Stack memory, used for function calls, local variables, and return addresses
- Shared libraries
- Memory-mapped files

The OS must make sure that:

- Each process gets enough memory to execute.
- One process cannot accidentally or maliciously access another process's memory.
- CPU memory accesses are fast.
- Memory is used efficiently.
- Programs can run even if their total memory requirement is larger than physical RAM.

Memory management is closely connected with the CPU, MMU, page table, TLB, disk, and process scheduler.

2. Logical Address and Physical Address

Logical Address

A logical address is the address generated by the CPU while a program is running.

It is also called:

- Virtual address
- Program address
- CPU-generated address

The program does not directly know where it is placed in physical RAM. It uses logical addresses, and the hardware plus OS translate them into real physical addresses.

Example:

```
Instruction: load value from address 2048
```

Here, 2048 is a logical address from the process's point of view.

Physical Address

A physical address is the actual address in RAM where data or instructions are stored.

It is the address seen by the memory unit.

Example:

```
Logical address 2048 may be translated to physical address 18432
```

The CPU produces the logical address. The memory hardware finally uses the physical address.

Why Do We Need Two Types of Addresses?

Logical and physical addresses are separated because:

- Programs should not need to know their actual RAM location.
- The OS can move processes in memory.
- Each process can have its own private address space.
- Memory protection becomes easier.
- Virtual memory becomes possible.
- Programs can be larger than physical RAM.

Address Binding

Address binding means mapping program addresses to actual memory addresses.

It can happen at three times:

1. Compile-Time Binding

If the memory location is known before execution, physical addresses can be generated at compile time.

Example:

```
Program will always start at physical address 1000.
```

Disadvantage:

- If the starting location changes, the program must be recompiled.
- Not flexible for modern systems.

2. Load-Time Binding

If the memory location is not known at compile time, the compiler generates relocatable code. Actual addresses are assigned when the program is loaded into memory.

Disadvantage:

- Once loaded, the process cannot easily move during execution.

3. Execution-Time Binding

The logical address is translated to a physical address while the program is running.

This requires hardware support, usually from the MMU.

Modern operating systems use execution-time binding.

Logical vs Physical Address

Feature	Logical Address	Physical Address
Generated by	CPU	MMU after translation
Also called	Virtual address	Real memory address
Seen by program	Yes	No
Seen by RAM	No	Yes
Same for all executions?	Can be same	May change
Used for	Process address space	Actual memory access

3. MMU: Memory Management Unit

The Memory Management Unit, or MMU, is a hardware component responsible for translating logical addresses into physical addresses.

It is usually part of the CPU.

Main Functions of MMU

The MMU performs:

- Address translation
- Memory protection
- Permission checking
- Page table lookup
- TLB lookup
- Support for virtual memory
- Support for paging and segmentation

Simple MMU Example Using Relocation Register

In a simple system, the MMU may use a relocation register.

```
Physical address = Logical address + Relocation register
```

Example:

```
Logical address = 500  
Relocation register = 14000  
  
Physical address = 500 + 14000 = 14500
```

The program thinks it is accessing address 500, but RAM receives address 14500.

Base and Limit Register

A simple protection mechanism uses:

- Base register: starting physical address of the process
- Limit register: size/range of the process

For every memory access:

```
If logical address < limit:  
    physical address = base + logical address  
Else:  
    trap/error
```

Example:

```
Base = 10000  
Limit = 4000  
Logical address = 3000  
  
3000 < 4000, so access is valid.  
Physical address = 10000 + 3000 = 13000
```

Invalid example:

```
Base = 10000  
Limit = 4000  
Logical address = 4500  
  
4500 > 4000, so access is invalid.
```

Why MMU Is Important

Without MMU:

- A process could directly access any RAM location.
- Memory protection would be weak.
- Virtual memory would be difficult.
- Multitasking would be unsafe.

With MMU:

- Each process gets an isolated virtual address space.
- Programs can use logical addresses.
- The OS can map and protect memory flexibly.

4. Contiguous and Non-Contiguous Memory Allocation

Before studying paging and segmentation, it is useful to understand two broad memory allocation styles.

Contiguous Memory Allocation

In contiguous allocation, each process is stored in one continuous block of physical memory.

Example:

```
RAM:
+-----+-----+-----+
| Process A | Process B | Process C |
+-----+-----+-----+
```

Advantages:

- Simple to implement
- Fast address translation

Disadvantages:

- Causes external fragmentation
- Difficult to fit large processes
- Memory compaction may be required

Non-Contiguous Memory Allocation

In non-contiguous allocation, a process can be divided and stored in different parts of memory.

Example:

```
Process A parts may be stored at different RAM locations.
```

Paging and segmentation are examples of non-contiguous allocation.

Advantages:

- Better memory utilization
- Less need for large continuous free blocks
- Supports virtual memory better

Disadvantages:

- More complex address translation
- Requires tables such as page tables or segment tables

5. Paging

Paging is a memory management scheme where the logical address space is divided into fixed-size blocks called pages, and physical memory is divided into fixed-size blocks called frames.

Key Terms

Term	Meaning
Page	Fixed-size block of logical memory
Frame	Fixed-size block of physical memory
Page table	Data structure that maps pages to frames
Page number	Identifies a page in logical address
Offset	Position inside a page

Page size and frame size are always equal.

Example:

```
Page size = Frame size = 4 KB
```

Why Paging Is Used

Paging solves the problem of external fragmentation.

Since every frame has the same size, any page can be placed into any free frame.

Example:

```
Logical memory:
+-----+-----+-----+-----+
| Page0 | Page1 | Page2 | Page3 |
+-----+-----+-----+-----+

Physical memory:
```

```

+-----+-----+-----+-----+
| Frame5 | Frame2 | Frame9 | Frame1 | Frame6 |
+-----+-----+-----+-----+

```

Page table:

Page 0 -> Frame 5

Page 1 -> Frame 2

Page 2 -> Frame 9

Page 3 -> Frame 1

The process appears continuous logically, but it may be scattered physically.

Logical Address in Paging

A logical address is divided into:

$$\text{Logical address} = \text{Page number} + \text{Offset}$$

The page number is used to index the page table. The offset is copied into the physical address.

$$\text{Physical address} = \text{Frame number} + \text{Offset}$$

Paging Address Translation

Steps:

1. CPU generates a logical address.
2. Logical address is divided into page number and offset.
3. Page number is used to find the frame number from the page table.
4. Frame number is combined with offset.
5. Physical address is sent to RAM.

Diagram:

CPU logical address

|
v

```

+-----+
| Page | Offset |
+-----+

```

|
v

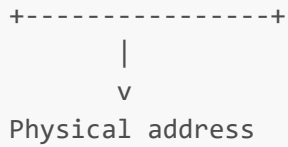
Page table lookup

|
v

```

+-----+
| Frame | Offset |

```



Physical address

Paging Numerical Example

Given:

Logical address space = 16 pages
Page size = 1 KB
Logical address = 2500

Since page size is 1 KB:

Page size = 1024 bytes
Page number = $2500 / 1024 = 2$
Offset = $2500 \% 1024 = 452$

If page table says:

Page 2 -> Frame 7

Then:

Physical address = Frame number * Frame size + Offset
Physical address = $7 * 1024 + 452$
Physical address = 7620

Internal Fragmentation in Paging

Paging removes external fragmentation, but it can cause internal fragmentation.

Internal fragmentation occurs when the last page of a process is not fully used.

Example:

Process size = 10 KB + 100 bytes
Page size = 4 KB

Number of pages needed:

3 pages = 12 KB

Wasted memory:

12 KB - (10 KB + 100 bytes)

This unused space inside the allocated page is internal fragmentation.

Advantages of Paging

- No external fragmentation
- Easy memory allocation
- Supports virtual memory
- Allows non-contiguous allocation
- Enables sharing of pages
- Enables memory protection per page

Disadvantages of Paging

- Page table requires extra memory
- Address translation takes time
- Internal fragmentation is possible
- TLB is needed for better performance

6. Page Table

A page table stores the mapping from virtual pages to physical frames.

Each process has its own page table.

Page Table Entry

Each entry in the page table is called a Page Table Entry, or PTE.

A PTE commonly contains:

- Frame number
- Valid/invalid bit
- Protection bits
- Dirty bit
- Reference bit
- Present bit
- Cache control bits

Important Bits in Page Table Entry

Valid/Invalid Bit

Indicates whether the page belongs to the process's valid address space.

```
Valid bit = 1 means page is valid.  
Valid bit = 0 means illegal access or not part of process.
```

Present Bit

Indicates whether the page is currently present in RAM.

```
Present = 1 means page is in RAM.  
Present = 0 means page is on disk or not loaded.
```

If the page is not present and the process tries to access it, a page fault occurs.

Protection Bits

Define allowed operations:

- Read
- Write
- Execute

Example:

```
Code page: read + execute  
Data page: read + write
```

If a process tries to write to a read-only page, a protection fault occurs.

Dirty Bit

Indicates whether the page has been modified after being loaded into memory.

```
Dirty = 1 means modified.  
Dirty = 0 means not modified.
```

If a dirty page is replaced, it must be written back to disk.

If a clean page is replaced, it can simply be discarded.

Reference Bit

Indicates whether the page has been accessed recently.

It is useful in page replacement algorithms such as Second Chance and LRU approximation.

7. TLB: Translation Lookaside Buffer

The Translation Lookaside Buffer, or TLB, is a small, fast cache inside the MMU that stores recently used page table entries.

It speeds up virtual-to-physical address translation.

Why TLB Is Needed

Without TLB, every memory access may require:

1. One memory access to read the page table entry.
2. One memory access to read/write the actual data.

So one logical memory access can become two physical memory accesses.

This is slow.

The TLB stores recent page-to-frame mappings so that page table lookup can often be skipped.

TLB Hit

A TLB hit occurs when the required page number is found in the TLB.

Steps:

1. CPU generates logical address.
2. MMU checks TLB.
3. Page number is found.
4. Frame number is obtained quickly.
5. Physical address is generated.

This is fast.

TLB Miss

A TLB miss occurs when the required page number is not found in the TLB.

Steps:

1. CPU generates logical address.
2. MMU checks TLB.
3. Page number is not found.
4. Page table is accessed from memory.
5. Frame number is found.
6. TLB is updated.
7. Physical address is generated.

This is slower than a TLB hit.

Effective Access Time With TLB

Common formula:

$$\text{EAT} = \text{Hit ratio} * \text{TLB hit time path} + \text{Miss ratio} * \text{TLB miss time path}$$

If:

```
Memory access time = m
TLB lookup time = t
Hit ratio = h
```

Then:

$$\text{EAT} = h(t + m) + (1 - h)(t + 2m)$$

Because:

- On TLB hit: TLB lookup + actual memory access
- On TLB miss: TLB lookup + page table memory access + actual memory access

TLB Numerical Example

Given:

```
TLB lookup time = 10 ns
Memory access time = 100 ns
TLB hit ratio = 90% = 0.9
```

```
EAT = 0.9(10 + 100) + 0.1(10 + 2 * 100)
EAT = 0.9(110) + 0.1(210)
EAT = 99 + 21
EAT = 120 ns
```

So the effective access time is **120 ns**.

TLB and Context Switching

Each process has its own page table. During a context switch, TLB entries from the old process may become invalid.

Solutions:

- Flush the TLB on every context switch.
- Use Address Space Identifiers, or ASIDs, to tag TLB entries by process.

Flushing is simple but reduces performance.

ASIDs are faster because entries from multiple processes can safely remain in the TLB.

8. Segmentation

Segmentation is a memory management scheme where a program is divided into variable-sized logical units called segments.

Unlike paging, which divides memory into fixed-size pages, segmentation divides a program according to logical structure.

Common Segments

A process may have segments such as:

- Code segment
- Data segment
- Stack segment
- Heap segment
- Library segment

Logical Address in Segmentation

In segmentation, a logical address contains:

$$\text{Logical address} = \text{Segment number} + \text{Offset}$$

The segment number is used to index the segment table.

Each segment table entry contains:

- Base address
- Limit
- Protection bits

Segment Table

Segment	Base	Limit
0: Code	1000	400
1: Data	3000	600
2: Stack	5000	300

If logical address is:

```
Segment number = 1  
Offset = 250
```

Then:

```
Offset 250 < Limit 600, so address is valid.  
Physical address = Base + Offset  
Physical address = 3000 + 250 = 3250
```

Invalid example:

```
Segment number = 2  
Offset = 350  
Limit = 300
```

Since **350** > **300**, this causes a segmentation fault.

Advantages of Segmentation

- Matches the programmer's logical view of a program
- Easy to share logical units such as code segments
- Protection can be applied segment-wise
- Different segments can have different permissions
- Supports dynamic growth of stack and heap

Disadvantages of Segmentation

- Causes external fragmentation
- Variable-size allocation is harder
- Compaction may be needed
- More complex than simple contiguous allocation

Paging vs Segmentation

Feature	Paging	Segmentation
Division type	Fixed-size pages	Variable-size segments
Based on	Physical memory management	Logical program structure
Address format	Page number + offset	Segment number + offset
Fragmentation	Internal fragmentation	External fragmentation
Programmer visibility	Usually invisible	Often visible logically
Table used	Page table	Segment table

Feature	Paging	Segmentation
Protection	Per page	Per segment

Segmentation With Paging

Some systems combine segmentation and paging.

In this approach:

- Program is divided into segments.
- Each segment is divided into pages.
- Segment table points to page tables.

This gives:

- Logical structure from segmentation
- Reduced external fragmentation from paging

However, it is more complex.

9. Virtual Memory

Virtual memory is a memory management technique that gives each process the illusion of having a large, continuous, private memory space, even if physical RAM is limited.

Virtual memory uses disk as an extension of RAM.

The disk area used for virtual memory is commonly called:

- Swap space
- Backing store
- Page file

Main Idea

Only the needed parts of a program are loaded into RAM. The remaining parts stay on disk until required.

Example:

```
Program size = 2 GB
Available RAM = 512 MB
```

The program can still run because not all 2 GB need to be in RAM at the same time.

Why Virtual Memory Is Useful

Virtual memory allows:

- Programs larger than physical RAM to run.

- Better degree of multiprogramming.
- Better CPU utilization.
- Process isolation.
- Easier memory sharing.
- Efficient program startup.
- Demand paging.

Virtual Address Space

Each process gets its own virtual address space.

Example:

```
Process A thinks it owns addresses 0 to  $2^{32} - 1$ .  
Process B also thinks it owns addresses 0 to  $2^{32} - 1$ .
```

But their virtual addresses map to different physical frames.

This prevents one process from directly accessing another process's memory.

Benefits of Virtual Memory

1. Isolation

Each process has a private address space. A bug in one process is less likely to corrupt another process.

2. Protection

Page table permissions can prevent invalid access.

Example:

```
Code pages can be marked read-only and executable.
```

3. Sharing

Multiple processes can share the same physical frames.

Example:

```
Shared library code can be mapped into many processes.
```

4. Efficient Loading

The OS does not need to load the entire program before execution starts.

5. Memory-Mapped Files

Files can be mapped into virtual memory and accessed like arrays.

10. Demand Paging

Demand paging is a virtual memory technique where a page is loaded into RAM only when it is actually needed.

The word "demand" means the page is brought into memory on demand, not in advance.

Basic Idea

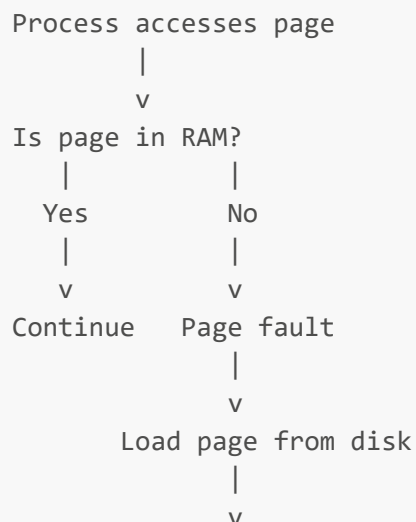
At program start:

- Not all pages are loaded.
- Page table entries for unloaded pages are marked not present.
- When the process accesses an unloaded page, a page fault occurs.
- The OS loads that page from disk into RAM.

Demand Paging Steps

1. CPU generates a logical address.
2. MMU checks the page table.
3. Present bit is 0.
4. A page fault trap occurs.
5. OS checks whether the address is valid.
6. If invalid, the process is terminated.
7. If valid, OS finds a free frame.
8. OS reads the required page from disk into the free frame.
9. Page table is updated.
10. The faulting instruction is restarted.

Demand Paging Diagram



```
Update page table
  |
  v
Restart instruction
```

Lazy Loading

Demand paging is a form of lazy loading.

The OS delays loading a page until it is actually referenced.

Advantages of Demand Paging

- Less initial loading time
- Less memory usage
- Allows large programs to run
- More processes can fit in memory
- Pages that are never used are never loaded

Disadvantages of Demand Paging

- Page faults are expensive
- Too many page faults can severely slow down the system
- Requires hardware support
- Requires backing store/disk support
- Page replacement may be needed

11. Page Fault

A page fault occurs when a process tries to access a page that is not currently present in physical memory.

It is not always an error. In demand paging, page faults are normal.

When Does a Page Fault Occur?

A page fault can occur when:

- The page is valid but currently on disk.
- The page has not been loaded yet.
- The process accesses memory outside its valid address space.
- The process violates memory protection.

The first two are normal demand paging faults. The last two are invalid faults and may terminate the process.

Page Fault Handling

Detailed steps:

1. CPU generates a virtual address.
2. MMU checks the page table.

3. Present bit is 0, so MMU raises a page fault trap.
4. Control transfers to the OS page fault handler.
5. OS checks if the reference is valid.
6. If invalid, OS terminates the process.
7. If valid, OS locates the page on disk.
8. OS finds a free physical frame.
9. If no free frame exists, OS selects a victim page using a page replacement algorithm.
10. If victim page is dirty, OS writes it back to disk.
11. OS loads the required page into the selected frame.
12. OS updates the page table and TLB if needed.
13. OS restarts the instruction that caused the fault.

Page Fault Service Time

Page fault handling is slow because disk I/O may be involved.

Approximate costs:

- TLB hit: nanoseconds
- RAM access: nanoseconds
- SSD access: microseconds
- HDD access: milliseconds

This means page faults are much more expensive than normal memory accesses.

Page Fault Rate

Page fault rate is the fraction of memory accesses that cause page faults.

If:

```
p = page fault rate
Memory access time = m
Page fault service time = f
```

Then:

```
Effective Access Time = (1 - p) * m + p * f
```

Because f is very large, even a small page fault rate can badly affect performance.

Page Fault Numerical Example

Given:

```
Memory access time = 100 ns
Page fault service time = 8 ms
Page fault rate = 0.001
```

Convert:

```
8 ms = 8,000,000 ns
```

```
EAT = (1 - 0.001) * 100 + 0.001 * 8,000,000
EAT = 0.999 * 100 + 8000
EAT = 99.9 + 8000
EAT = 8099.9 ns
```

Even a **0.1%** page fault rate increases access time massively.

12. Page Replacement

If a page fault occurs and no free frame is available, the OS must remove some page from RAM. This is called page replacement.

The removed page is called the victim page.

Page Replacement Steps

1. Page fault occurs.
2. OS checks that the requested page is valid.
3. OS finds no free frame.
4. OS chooses a victim page.
5. If victim page is dirty, write it back to disk.
6. Load required page into the freed frame.
7. Update page table.
8. Restart instruction.

Common Page Replacement Algorithms

FIFO: First-In First-Out

The oldest loaded page is replaced first.

Advantages:

- Simple

Disadvantages:

- May replace a heavily used page
- Can suffer from Belady's anomaly

Optimal Page Replacement

Replaces the page that will not be used for the longest time in the future.

Advantages:

- Gives the minimum possible page faults

Disadvantages:

- Impossible to implement perfectly because future references are unknown
- Used mainly for comparison

LRU: Least Recently Used

Replaces the page that has not been used for the longest time.

Advantages:

- Good practical performance

Disadvantages:

- Exact implementation is expensive
- Requires hardware or software tracking

Second Chance

A practical approximation of LRU.

It uses the reference bit.

Rule:

- If reference bit is 0, replace the page.
- If reference bit is 1, clear it and give the page another chance.

13. Thrashing

Thrashing occurs when the system spends more time handling page faults than executing actual instructions.

It usually happens when too many processes are loaded and each process does not get enough frames.

Symptoms of Thrashing

- Very high page fault rate
- Low CPU utilization
- Heavy disk activity
- System becomes extremely slow

Cause

Each process needs a minimum number of pages in memory to run efficiently. This set is called its working set.

If the OS gives a process fewer frames than its working set, the process keeps faulting again and again.

How to Reduce Thrashing

- Reduce degree of multiprogramming
 - Allocate more frames to processes
 - Use working set model
 - Use page fault frequency control
 - Add more RAM
 - Improve locality of reference in programs
-

14. Locality of Reference

Locality of reference is the tendency of programs to access a relatively small portion of memory repeatedly over a short period.

Virtual memory works well because programs usually show locality.

Temporal Locality

If a memory location is accessed now, it is likely to be accessed again soon.

Example:

```
for (int i = 0; i < 1000; i++) {  
    sum += x;  
}
```

Variable `x` is accessed repeatedly.

Spatial Locality

If a memory location is accessed, nearby memory locations are likely to be accessed soon.

Example:

```
for (int i = 0; i < 1000; i++) {  
    sum += arr[i];  
}
```

Array elements are stored close together, so nearby addresses are accessed.

15. Complete Flow: From Virtual Address to Data

This is a common interview explanation.

Suppose a process executes:

```
x = arr[i];
```

The CPU must access memory.

Steps:

1. CPU generates a virtual address for `arr[i]`.
2. MMU splits the address into page number and offset.
3. MMU checks the TLB.
4. If TLB hit, frame number is found quickly.
5. If TLB miss, page table is checked in memory.
6. If page table says the page is present, frame number is obtained.
7. If page table says the page is not present, page fault occurs.
8. OS handles the page fault and loads the page from disk.
9. Page table and TLB are updated.
10. Frame number and offset form the physical address.
11. RAM is accessed.
12. Data is returned to CPU.

Short version:

```
CPU virtual address  
-> TLB lookup  
-> page table if TLB miss  
-> page fault if page absent  
-> physical address  
-> RAM access
```

16. Important Formulas

Paging Address Calculation

```
Page number = Logical address / Page size  
Offset = Logical address % Page size  
Physical address = Frame number * Frame size + Offset
```

Number of Pages

$$\text{Number of pages} = \text{Process size} / \text{Page size}$$

If not exactly divisible, round up.

$$\text{Number of pages} = \text{ceil}(\text{Process size} / \text{Page size})$$

Number of Frames

$$\text{Number of frames} = \text{Physical memory size} / \text{Frame size}$$

Bits for Offset

$$\text{Offset bits} = \log_2(\text{Page size})$$

Example:

Page size = 4 KB = 4096 bytes = 2^{12} bytes
Offset bits = 12

Bits for Page Number

$$\text{Page number bits} = \text{Logical address bits} - \text{Offset bits}$$

Bits for Frame Number

$$\text{Frame number bits} = \text{Physical address bits} - \text{Offset bits}$$

TLB Effective Access Time

$$\text{EAT} = h(t + m) + (1 - h)(t + 2m)$$

Where:

- h = TLB hit ratio
- t = TLB lookup time

- m = memory access time

Page Fault Effective Access Time

$$\text{EAT} = (1 - p) * \text{memory access time} + p * \text{page fault service time}$$

Where:

- p = page fault rate

17. Common Interview Questions

1. What is the difference between logical and physical address?

Logical address is generated by the CPU and seen by the program. Physical address is the actual RAM address generated after MMU translation.

2. What is the role of MMU?

The MMU translates logical addresses to physical addresses, checks permissions, supports paging, uses the TLB, and enables virtual memory.

3. What is paging?

Paging divides logical memory into fixed-size pages and physical memory into equal-sized frames. A page table maps pages to frames.

4. What problem does paging solve?

Paging removes external fragmentation and allows non-contiguous memory allocation.

5. Does paging have fragmentation?

Yes. Paging can cause internal fragmentation, especially in the last page of a process.

6. What is segmentation?

Segmentation divides a program into variable-sized logical parts such as code, data, stack, and heap.

7. Paging vs segmentation?

Paging is fixed-size and mainly physical-memory oriented. Segmentation is variable-size and matches the logical structure of a program. Paging causes internal fragmentation. Segmentation causes external fragmentation.

8. What is TLB?

TLB is a small fast cache that stores recent page table entries to speed up address translation.

9. What happens on a TLB miss?

The MMU or OS checks the page table in memory, gets the frame number, updates the TLB, and continues address translation.

10. What is virtual memory?

Virtual memory gives each process a large private address space and allows execution even when the whole process is not loaded in RAM.

11. What is demand paging?

Demand paging loads a page into RAM only when the process actually accesses it.

12. What is a page fault?

A page fault occurs when a process accesses a page that is not currently present in RAM.

13. Is page fault an error?

Not always. In demand paging, a valid page fault is normal. Invalid memory access or permission violation is an error.

14. Why are page faults expensive?

Because they may require disk I/O, page replacement, page table updates, and restarting the faulting instruction.

15. What is thrashing?

Thrashing occurs when excessive page faults make the system spend more time swapping pages than executing processes.

18. Quick Revision Table

Topic	One-Line Meaning
Logical address	Address generated by CPU
Physical address	Actual address in RAM
MMU	Hardware that translates logical to physical address
Paging	Fixed-size virtual pages mapped to physical frames
Page table	Maps page numbers to frame numbers
TLB	Fast cache for page table entries
Segmentation	Variable-size logical division of program
Virtual memory	Illusion of large private memory using RAM plus disk

Topic	One-Line Meaning
Demand paging	Load pages only when needed
Page fault	Required page is not present in RAM
Internal fragmentation	Wasted space inside allocated block
External fragmentation	Free memory exists but is scattered
Thrashing	Too many page faults, very low useful work

19. Final Mental Model

Think of memory management like this:

```
Program uses virtual/logical addresses.  
CPU generates virtual address.  
MMU translates virtual address to physical address.  
TLB makes translation faster.  
Page table stores page-to-frame mapping.  
Paging allows non-contiguous memory allocation.  
Segmentation divides memory according to program logic.  
Virtual memory allows programs to run without fully being in RAM.  
Demand paging loads pages only when needed.  
Page fault happens when needed page is absent from RAM.
```

If you can explain this flow clearly, you can answer most placement-level memory management questions.